8085 Programming

Before starting to write programs, learn the instruction set in 8085.

🔹 1. Data Transfer Instructions

Used to move data between registers, memory, and I/O.

Mnemonic	Meaning	Use Example
MVI R, data	Move immediate to register	MVI A, 32H (A = 32H)
MOV R1, R2	Copy data from R2 to R1	MOV B, C (B = C)
LDA addr	Load A from memory	LDA 2050H (A = M[2050])
STA addr	Store A to memory	STA 3050H (M[3050] = A)
LHLD addr	Load H-L pair	LHLD 2500H (L = M[2500], H = M[2501])
SHLD addr	Store H-L pair	SHLD 2700H
XCHG	Exchange DE and HL	HL ⇔ DE

✅ **Use MVI, MOV a lot** in setting up your program.

🔹 2. Arithmetic Instructions

Perform addition, subtraction, increment, decrement.

Mnemonic	Meaning	Use Example
ADD R	A = A + R	ADD B
ADI data	A = A + data	ADI 05H
SUB R	A = A - R	SUB C
SUI data	A = A - data	SUI 01H
INR R	R = R + 1	INR A
DCR R	R = R - 1	DCR C

INX RP	Increment Register Pair	INX H
DCX RP	Decrement Register Pair	DCX D

✅ **Use INR/DCR for loops and counters.**

🔹 3. Logical Instructions

Bitwise operations and comparisons.

Mnemonic	Meaning	Use Example
ANA R	A = A & R	ANA C
ORA R	A = A	R
XRA R	A = A ^ R	XRA D
CMA	Complement A	CMA (A = ~A)
CMP R	Compare A and R	CMP B (sets flags only)

✅ **Use CMP for decisions in JZ, JNZ, etc.**

🔹 4. Branching Instructions

Conditional and unconditional jumps, used for loops and decision-making.

Mnemonic	Meaning	Use Example
JMP addr	Jump unconditionally	JMP 3000H
JC addr	Jump if carry	JC 3100H
JNC addr	Jump if no carry	
JZ addr	Jump if zero	JZ 3200H
JNZ addr	Jump if not zero	
CALL addr	Call subroutine	CALL 4000H
RET	Return from subroutine	

✅ **You will use JZ, JNZ, JMP all the time.**

🔹 5. Stack Instructions

Used for storing temporary data or function calls.

Mnemonic	Meaning	Use Example
PUSH RP	Push register pair	PUSH B
POP RP	Pop into register pair	POP D
XTHL	Exchange top of stack with HL	
SPHL	Move HL to SP	

✅ **Used in subroutine or nested function calls.**

🔹 6. I/O Instructions

For dealing with ports and peripherals.

Mnemonic	Meaning	Use Example
IN port	Read from port	IN 01H
OUT port	Write to port	OUT 02H

🔹 7. Control Instructions

To control execution flow or system status.

Mnemonic	Meaning
NOP	Do nothing
HLT	Halt execution
DI	Disable interrupts
EI	Enable interrupts

✅ HLT is used to stop the program cleanly.

🔹 Most Common Instruction Pattern (Typical Loop)

```
MVI C, 05H   ; Load counter
LOOP:
  INR A      ; Increment A
  DCR C      ; Decrease counter
  JNZ LOOP   ; Repeat if not zero
HLT          ; End
```

👉 Summary: Must-Know Mnemonics

```
MVI, MOV, LDA, STA
ADD, SUB, INR, DCR
CMP, JMP, JZ, JNZ
CALL, RET
PUSH, POP
IN, OUT
HLT
```

Practice Questions

Write a program in 8085 to add two 8 bit numbers

✅ Problem:

Add two 8-bit numbers stored at memory locations 2000H and 2001H.
Store the result at 2002H, and if there's a carry, store it at 2003H.

🔧 8085 Assembly Program:

```
LXI H, 2000H   ; Load HL pair with address 2000H
MOV A, M       ; Load A with the first number (M = [2000H])
INX H          ; Point HL to next address (2001H)
ADD M          ; Add second number (M = [2001H]) to A
INX H          ; Point HL to next address (2002H)
MOV M, A       ; Store result in 2002H
INX H          ; Point to 2003H for carry
MOV A, 00H     ; Clear A
ACI 00H        ; Add immediate with carry (to get CY into A)
MOV M, A       ; Store carry (0 or 1) in 2003H
HLT            ; Stop
```

🧠 Explanation:

- LXI H, 2000H: Load address of the first number.
- MOV A, M: Move first number to accumulator.
- INX H: Move to second number.
- ADD M: Add second number to A (result in A, carry in flag).
- INX H: Point to result location.
- MOV M, A: Store result.

- ACI 00H: Add 0 + CY flag → puts carry into A.
- MOV M, A: Store carry (0 or 1).
- HLT: End of program.

🧪 Example:

If
M[2000H] = 5AH (90)
M[2001H] = 78H (120)

Then:
Result = 5AH + 78H = D2H (210), no carry →

- M[2002H] = D2H
- M[2003H] = 00H

If
M[2000H] = FFH (255)
M[2001H] = 02H (2)

Then:
Result = 01H with carry →

- M[2002H] = 01H
- M[2003H] = 01H

Program to check if a number is odd or even

✅ Problem:

Check if the number at memory location 2000H is **odd or even**.

- If **even**, store 00H at 2001H
- If **odd**, store 01H at 2001H

🔧 8085 Program:

```
LDA 2000H       ; Load number from memory into A
ANI 01H         ; AND with 01H → result is 1 if odd, 0 if even
STA 2001H       ; Store result (00 or 01) in 2001H
HLT             ; End
```

🧠 Explanation:

- ANI 01H performs a bitwise AND with 00000001 → only LSB (least significant bit) matters.

- If LSB is 1, number is odd → result = 01H
- If LSB is 0, number is even → result = 00H

🧪 Example:

- If M[2000H] = 2AH (42):
  → LSB = 0 → even
  → M[2001H] = 00H

- If M[2000H] = 37H (55):
  → LSB = 1 → odd
  → M[2001H] = 01H

Program to find highest number in a series or array

✅ Problem:

An array of N 8-bit numbers starts at memory location 3001H.

- First byte at 3000H holds N (the count).
- Your task is to find the **maximum number** and store it at 3000 + N + 1 (or say result_addr).

🔧 8085 Program:

```
LXI H, 3000H     ; HL points to start of data
MOV C, M         ; C = count (N)
INX H            ; Point to first element
MOV A, M         ; Assume first number is max
DCR C            ; Decrement count (already took first number)
LOOP: INX H      ; Move to next number
CMP M            ; Compare A with M
JC SKIP          ; If A &lt; M, skip update
MOV A, M         ; A = new maximum
SKIP: DCR C      ; Decrease count
JNZ LOOP         ; Repeat if count ≠ 0
INX H            ; Move to next location to store result
MOV M, A         ; Store maximum value
HLT              ; Stop
```

🧠 Explanation:

- First, load count N into register C.
- Load first number into accumulator A (assume it's max).
- Loop through remaining numbers:

    - Compare with A.
    - If current > A, update A.
- After loop, A has the max value.
- Store the result after the last element.

🧪 Example:

Memory contents:

```
3000H: 05H        ; N = 5
3001H: 23H
3002H: 87H
3003H: 52H
3004H: A3H
3005H: 66H
```

After program runs:

```
3006H: A3H        ; Maximum value
```

💡 Notes:

- Uses simple CMP, JC, and loop control.
- JC = Jump if Carry → means A < M, so don't update.
- No flags or stack tricks here — clean logic.

Program to find smallest number in array

✅ Problem:

- Array of N elements starts at memory location 3001H
- 3000H holds the count N
- You need to find the **smallest number** and store it at 3000H + N + 1

🔧 8085 Program:

```
LXI H, 3000H     ; HL points to base address
MOV C, M         ; C = N (count)
INX H            ; HL = 3001H (first number)
MOV A, M         ; Assume first number is the minimum
DCR C            ; Decrease count (first number already taken)
LOOP: INX H      ; Move to next number
```

```
CMP M            ; Compare A with current
JNC SKIP         ; Jump if A &lt;= M (A is smaller, skip update)
MOV A, M         ; Update A to new minimum
SKIP: DCR C      ; Decrease counter
JNZ LOOP         ; Repeat if count ≠ 0
INX H            ; Move to result location
MOV M, A         ; Store minimum
HLT              ; Stop
```

🧠 Explanation:

- Start with assuming first element is the minimum.
- Loop through the array:
  - If any number is **smaller**, update A.
- At end, A holds the smallest number.

🧪 Example Memory:

Address	Value
3000H	04H
3001H	55H
3002H	12H
3003H	87H
3004H	39H

✅ Result:

- Minimum = 12H
- Stored at 3005H

🔁 Difference from max-finding version:

- We use JNC (Jump if No Carry), i.e., A ≤ M, so skip updating.

Program for addition of BCD numbers

To perform **BCD (Binary Coded Decimal)** addition in 8085, the key is to use the **DAA (Decimal Adjust Accumulator)** instruction **after binary addition** to correct the result into valid BCD.

✅ Problem:

Add two BCD numbers:

- First BCD number in register A
- Second BCD number in register C
- Store the **BCD-corrected result** in memory at 3000H
- If there is a **carry**, store it at 3001H

🔧 8085 Assembly Program:

```
MVI A, 45H       ; Load first BCD number (e.g. 45)
MVI C, 38H       ; Load second BCD number (e.g. 38)
ADD C            ; Add them (binary addition first)
DAA              ; Adjust result to valid BCD
LXI H, 3000H     ; HL = result memory location
MOV M, A         ; Store BCD result at 3000H
INX H            ; Point to 3001H
MOV A, 00H       ; Clear A
ACI 00H          ; Add CY (if any) into A
MOV M, A         ; Store carry (0 or 1) at 3001H
HLT              ; Stop
```

🧠 How it works:

- ADD C: Adds binary values.
- DAA: Converts binary result to correct BCD (if lower nibble > 9 or carry).
- ACI 00H: Adds carry flag into accumulator (sets A = 01H if there was carry).
- Result and carry are stored.

🧪 Example:

Input:

- A = 45H (BCD = 45)
- C = 38H (BCD = 38)
  → 45 + 38 = 83 (in BCD: 83H)

Binary ADD:

- 45H + 38H = 7DH
- DAA adjusts 7D → 83H (valid BCD)

Final memory:

- M[3000H] = 83H
- M[3001H] = 00H (no carry)

📌 Important Notes:

- BCD numbers must be **properly encoded** (i.e., 45 means 0100 0101, not 2D).
- Use DAA **only after binary addition**.
- Only one BCD number should be in A; the other in another register.

Program to subtract BCD Number

Subtracting **BCD numbers** in 8085 is a bit trickier than addition, because **8085 has no DAA equivalent for subtraction** — there's **no DAS (Decimal Adjust for Subtraction)** like in some other CPUs.

However, we can still do **BCD subtraction using binary subtraction**, **then manually adjust** the result **if needed**.

⚠️ Before You Begin:

8085 does **binary subtraction** only — it doesn't automatically correct BCD subtraction results. So we typically:

- Perform normal binary subtraction.
- Skip DAA (because DAA is **not valid for subtraction**).
- Interpret the result *as long as the inputs and result are valid BCD*.

✅ Problem:

Subtract two valid 2-digit BCD numbers:

- First BCD number in register A
- Second BCD number in register C
- Store the result in memory at 3000H
- If there is a borrow, store 01H at 3001H, else 00H

🔧 8085 Program (BCD Subtraction):

```
MVI A, 79H        ; Load first BCD number (e.g., 79)
MVI C, 25H        ; Load second BCD number (e.g., 25)
SUB C             ; A = A - C (binary subtraction)
LXI H, 3000H      ; HL = result memory address
MOV M, A          ; Store the binary result (still looks like BCD if valid)
INX H             ; Next memory for borrow
MOV A, 00H        ; Clear A
SBB A             ; A = 00 - borrow (if borrow occurred CY = 1 → A = FF)
ADI 01H           ; A = FF + 01 = 00 (if no borrow), A = 00 + 01 = 01 (if borrow)
```

```
MOV M, A          ; Store borrow info (00 or 01)
HLT
```

🧠 How It Works:

- SUB C: Subtracts second BCD from first.
- If borrow occurs, carry flag is set.
- We use SBB A after clearing A to get CY into A.
- ADI 01H: Turn CY into a 01H so we can store it as a value.

🧪 Example:

Case 1: No borrow

- A = 79H
- C = 25H
- 79 - 25 = 54
- Result: M[3000H] = 54H, M[3001H] = 00H

Case 2: With borrow

- A = 23H
- C = 47H
- 23 - 47 causes borrow → result = (2's complement)
- A will hold binary result = DC (≠ valid BCD)
- M[3000H] = DCH, M[3001H] = 01H (borrow occurred)

⚠️ Warning:

If the result **goes negative**, it will be a 2's complement result — **not a valid BCD**. You'll need more complex logic to represent negative BCD properly (like 10's complement BCD arithmetic, which is rarely used in 8085 level).

✅ Summary:

- Use SUB for BCD subtraction.
- Do **not use DAA** after subtraction.
- Store the result and whether borrow occurred.
- Be cautious with invalid BCD results if borrow occurs.

Conversion of Hexadecimal to BCD

✅ Problem:

Convert an 8-bit **hexadecimal number** (e.g., 5A = 90 decimal) to **BCD format**
Store:

- **Hundreds** digit at 3001H
- **Tens** digit at 3002H
- **Units** digit at 3003H

Hex number is at 3000H.

⚙️ 8085 Has No DIV Instruction, So:

We simulate division using **repeated subtraction** to isolate hundreds, tens, and units.

🔧 8085 Program:

```
LXI H, 3000H     ; HL points to 3000H
MOV A, M         ; Load hex number into A

MVI B, 00H       ; B = Hundreds
MVI C, 00H       ; C = Tens

HUNDRED_LOOP:
CPI 64H          ; Compare A with 100 (64H)
JC TEN_LOOP      ; If A &lt; 100, go to tens
SUI 64H          ; A = A - 100
INR B            ; B = B + 1 (hundreds count)
JMP HUNDRED_LOOP

TEN_LOOP:
CPI 0AH          ; Compare A with 10
JC STORE         ; If A &lt; 10, go to store
SUI 0AH          ; A = A - 10
INR C            ; C = C + 1 (tens count)
JMP TEN_LOOP

STORE:
LXI H, 3001H     ; Store from 3001H
MOV M, B         ; Store hundreds
INX H
MOV M, C         ; Store tens
INX H
MOV M, A         ; Store units
HLT
```

🧠 Explanation:

- We subtract 64H (100 decimal) repeatedly to count how many 100s fit — store in B.
- Then subtract 0AH (10 decimal) repeatedly — store in C.
- Whatever remains in A is the units digit.
- Final BCD is stored as 3 separate nibbles (e.g. 0-9 each) at 3001H, 3002H, 3003H.

🧪 Example:

Suppose M[3000H] = 5AH (hex)
→ 5A = 90 decimal

- 100s = 0 → B = 00H
- 10s = 9 → C = 09H
- 1s = 0 → A = 00H

✅ Final memory:

- 3001H = 00H
- 3002H = 09H
- 3003H = 00H

📌 Note:

This program works **only for 8-bit values (0–255)**, which fits perfectly in 8085's register capacity.

Conversion of BCD to HEX

Great — now let's reverse the operation and **convert a BCD number to its hexadecimal equivalent** in 8085.

✅ Problem:

You have a **3-digit BCD number** stored as:

- **Hundreds** digit at 3001H
- **Tens** digit at 3002H
- **Units** digit at 3003H

Each location contains one nibble as a byte (e.g., 05H, 07H, 09H).
You need to convert this to a single **hexadecimal number** and store it at 3000H.

🔧 8085 Assembly Program:

```
LXI H, 3001H      ; HL = 3001H (hundreds digit)
MOV B, M          ; B = Hundreds
INX H
```

```
MOV C, M          ; C = Tens
INX H
MOV D, M          ; D = Units

; Calculate B * 100
MVI A, 0          ; A = 0
MVI E, 0          ; E = Temporary accumulator

H_LOOP:
DCR B
JM T_LOOP         ; Jump if B &lt; 0
ADI 64H           ; A += 100 (64H)
JNC $+3           ; If carry not set, continue
INR E             ; Overflow to E
JMP H_LOOP

T_LOOP:
; A:E now holds B * 100

; Add C * 10
MVI H, 0
MVI L, 0
MOV B, C

TENS_LOOP:
DCR B
JM ADD_UNITS
ADI 0AH           ; A += 10
JNC $+3
INR E
JMP TENS_LOOP

ADD_UNITS:
ADD D             ; A = A + units
JNC $+3
INR E

; Now A:E holds the total in hex (BCD → binary)
; We&#39;ll just store A at 3000H (ignore overflow E for 8-bit result)

LXI H, 3000H
MOV M, A
HLT
```

🧠 **Explanation:**

- Multiply hundreds by 100 (64H), tens by 10 (0AH), and add units.
- Accumulator A builds the result.
- If A overflows (e.g., > FFH), we increment a temp register (E) — but we only store 8-bit result.

🧪 Example:

If memory:

```
3001H = 05H   ; Hundreds = 5
3002H = 06H   ; Tens     = 6
3003H = 09H   ; Units    = 9
```

Decimal = 5*100 + 6*10 + 9 = 569
Hex = 239H

A = 39H, E = 02H
→ Store lower byte 39H at 3000H

You can modify it to store both A and E for full 16-bit result if needed.

⚠️ Limitations:

- Only valid for BCD values (each digit ≤ 9).
- This version gives only **lower 8 bits**, i.e. max value = 255. You can expand to handle full 16-bit result if needed.

Program to multiply two number

Assumptions:

- Multiplicand at memory 2000H
- Multiplier at memory 2001H
- Result stored at 2002H (low byte) and 2003H (high byte)

```
LXI H, 2000H       ; HL points to multiplicand
MOV A, M           ; Load multiplicand into A
MOV B, A           ; Store multiplicand in B

INX H              ; HL points to multiplier
MOV A, M           ; Load multiplier into A
MOV C, A           ; Store multiplier in C

MVI D, 00H         ; Clear D (high byte of result)
```

```
MVI E, 00H          ; Clear E (low byte of result)

MOV A, 00H          ; Clear accumulator (use as temp sum low byte)
MOV L, A            ; Clear L (low byte of result)
MOV H, A            ; Clear H (high byte of result)

MUL_LOOP:
    MOV A, C
    CPI 00H
    JZ MUL_DONE      ; If multiplier zero, done

    ; Add multiplicand (B) to result (H:L)

    MOV A, L
    ADD B
    MOV L, A

    MOV A, H
    ADC 00H          ; Add carry from previous addition
    MOV H, A

    DCR C            ; Decrement multiplier
    JMP MUL_LOOP

MUL_DONE:
    LXI H, 2002H     ; HL points to result storage
    MOV M, L         ; Store low byte
    INX H
    MOV M, H         ; Store high byte

    HLT              ; End program
```

Program to divide two numbers

✅ Problem:

Divide the **dividend** by the **divisor**

- Dividend at memory 2000H
- Divisor at memory 2001H
- Store **quotient** at 2002H
- Store **remainder** at 2003H

🔧 8085 Assembly Program (Division by Repeated Subtraction):

```
LXI H, 2000H        ; HL points to dividend
MOV A, M            ; Load dividend in A
MOV B, A            ; Save dividend in B (remainder in B)

INX H               ; HL points to divisor
MOV C, M            ; Load divisor in C

MVI D, 00H          ; D = quotient (clear)

DIV_LOOP:
    MOV A, B        ; Load current remainder in A
    CMP C           ; Compare remainder with divisor
    JC DIV_DONE     ; If remainder &lt; divisor, done

    SUB C           ; remainder = remainder - divisor
    MOV B, A        ; Update remainder in B
    INR D           ; Increment quotient
    JMP DIV_LOOP

DIV_DONE:
    LXI H, 2002H
    MOV M, D        ; Store quotient
    INX H
    MOV M, B        ; Store remainder

    HLT
```

🧠 Explanation:

- Initialize quotient (D) to 0.
- While remainder (B) ≥ divisor (C): subtract divisor from remainder, increment quotient.
- When remainder < divisor, stop.
- Store quotient and remainder in memory.

🧪 Example:

- Dividend = 37H (55 decimal)
- Divisor = 0AH (10 decimal)
- Quotient = 05H (5)
- Remainder = 07H (7)